

Distilling a Classical Wordle Solver into a 0.5B Language Model

Eleven experiments on where the remaining gap actually lives

Technical report | Qwen2.5-0.5B-Instruct | August 2026
Source and reproduction: github.com/Arnavvs/Qwen_wordle_sft

Abstract

A classical constraint-and-information-theoretic Wordle solver plays the game in 3.4431 guesses on a 246-word held-out set. An untrained Qwen2.5-0.5B-Instruct cannot play it at all, failing 91-97% of games under every prompt format tried. We distil the classical solver into that model by supervised fine-tuning on 19,212 expert state-action pairs and decode under a feedback-consistency constraint, reaching **3.7642 guesses, 242/246 solved, 1.63% failure** - ahead of a classical random-elimination baseline (4.0203) and a letter-frequency heuristic (3.7927), and 0.3211 guesses short of the classical expert.

The substantive contribution is negative and diagnostic. Most of the distilled system's performance comes from the decoder, not the training: restricting guesses to feedback-consistent words is worth 1.54 guesses, more than everything learned in three preceding training phases combined. We then attempt to close the residual 0.3211 four times and fail each time, and explain why with a measurement we call the *decision budget*: pricing all 922 of the model's real decisions against the optimal action at each state shows that perfect action selection across the entire regime the decoder restricts is worth only **0.0698 guesses**, because the model is already optimal in 85.6% of those decisions and 83% of the remainder are exact ties. Direct preference optimisation loses 0.1708 to policy drift; group-relative policy optimisation on exactly-computed rewards moves the mean by 0.0041 (paired $t = -0.28$, 233 of 246 games byte-identical). Both outcomes were predicted by the budget before either run. A prompt-format crossover further shows that a large apparent prompt effect (+0.5163 guesses) is mostly format lock-in rather than a property of the format. We conclude that the residual gap is a capability limit in word retrieval, not an action-selection problem, and that the binding constraint is the expert's own scarcity of endgame demonstrations.

1. Introduction

Wordle is an unusually clean testbed for asking how much of an algorithm's competence can be transferred into a language model. The rules are trivial, the state is fully observable, an optimal or near-optimal reference policy is computable, and performance is a single scalar - the mean number of guesses. That combination makes it possible to ask not merely whether distillation works, but precisely which component of a distilled system is responsible for its score.

This report documents eleven experiments run in sequence, each designed to test the interpretation the previous one suggested. Several of those interpretations turned out to be wrong, and the corrections are retained in the text rather than quietly replaced, because the pattern of wrong readings is itself the finding: at three separate points the natural explanation for a result was contradicted by a control that had not yet been run.

Three results are worth stating up front. First, the decoder dominates: constraining generation to words consistent with observed feedback contributes more than all supervised training. Second, the residual gap between the distilled model and the classical expert is not an action-selection problem, and we can say so quantitatively rather than by elimination. Third, an apparently large prompt-format effect is mostly an artefact of what the model was trained on, and separating the two required training a second model rather than reasoning about the first.

2. Task, data, and evaluation protocol

2.1 The game and the vocabulary

A player has six attempts to identify a hidden five-letter word. Each guess returns five tiles: green for a correct letter in the correct position, yellow for a letter present elsewhere, grey for a letter absent or whose copies are already accounted

for. Duplicate letters follow the standard two-pass rule - greens are assigned first, and each yellow then draws from the remaining unmatched copies. We use a 2,315-word answer list and a 12,972-word list of legal guesses.

Two derived sets recur throughout and must not be confused. The *candidate* set is the answers still consistent with all feedback; it uses the privileged 2,315-word answer list and is never shown to the model. The *admissible* set is the legal guesses still consistent with all feedback - a pure function of the visible board and the public word list, and exactly what a hard-mode player computes for themselves. All bucketing in this report is keyed on the size of the admissible set, because that is the decision regime the deployed system actually faces.

2.2 Splits and leakage control

Answers are partitioned by a salted hash of the answer string into 2,069 training and 246 held-out words, identically across every experiment. All reported game results are on the same 246. Prompts are rendered by a single function that takes no answer parameter, so answer leakage is structurally impossible rather than merely audited; the candidate count and candidate list are likewise withheld, since both are solver-side quantities a player cannot see. Confirmed letters are rendered position by position rather than concatenated, because once all five are known the concatenation would be the answer.

2.3 Statistical protocol

The unpaired standard error of a mean over 246 games is roughly 0.064 guesses, which cannot resolve the effect sizes post-training typically produces. Every comparison in this report is therefore **paired** on identical answers with identical seeds, and reported as a paired t statistic over per-game differences. Where several variants are compared at once, we say so and decline to treat the best of them as significant on that basis alone.

One further protocol rule proved load-bearing. Every phase scores through the same evaluation harness, which begins by re-running a control condition that must reproduce 3.7642 before any other number in the run is read. An earlier result was voided precisely because a variant re-implemented part of the measurement path; the control exists to catch that class of error.

3. The classical expert

The reference policy is a symbolic solver with no learned component. It maintains the exact candidate set by table lookup against a precomputed 12,972 x 2,315 feedback matrix, and selects guesses by one of several decision rules over the induced partition of the candidate set.

Solver	Opener	Mean	Fail
random	-	4.0259	1.56%
frequency	STARE	3.7459	1.38%
entropy	SOARE	3.4644	0%
expected remaining	ROATE	3.4812	0%
minimax	ARISE	3.5732	0%
hybrid	RAISE	3.4812	0%
depth-6 lookahead	SOARE	3.4300	0%
depth-6 lookahead	SALET	3.4212	0%

Table 1. Classical solvers over the full 2,315-answer benchmark, guess pool = all legal words, six-guess limit. The solver reproduces published figures for this vocabulary (SOARE at 5.886 bits, ROATE at 60.42 expected remaining, minimax worst case 168), which is external validation that the feedback function and entropy computation are correct.

The distinction between the greedy rules and the lookahead search matters for interpreting everything that follows: greedy entropy sits roughly 0.03 guesses above true depth-6 search, so 'the classical expert' is a strong but not optimal reference.

4. Distillation

4.1 Trajectories and supervised data

Three expert policies were rolled out over all 2,315 answers, recording every state-action step. Two of the three share an opener with a third deliberately: comparing entropy against a lookahead solver that opens with the same word isolates the decision rule, while comparing two lookahead solvers with different openers isolates the opening word, which accounts for 77% of the gap between them.

Steps become supervised examples whose prompt contains the guess history, the constraints derivable from it, and a turn counter; the completion is the expert's next guess, with loss on the completion only. This yields 7,067-7,173 training rows per policy.

4.2 The first training run, and its failure

A LoRA adapter (rank 16, alpha 32, dropout 0.05, all seven projection modules) was trained on Qwen2.5-0.5B-Instruct at learning rate $2e-4$ with a cosine schedule, two epochs, effective batch 16, fp16, sequence length 640. Final training loss fell from about 4.7 to 0.481-0.508 across the three policies, with zero truncated examples. Evaluation used greedy decoding, strict first-token parsing, and the rule that an invalid guess consumes the turn and returns no feedback.

The result was a 76-79% failure rate. Loss had fallen, the model had plainly learned something, and it still could not finish games. Diagnosis occupied the next two phases.

4.3 What the model had and had not learned

The model reproduces its expert's opener 100% of the time and agrees with its turn-2 choice 90% of the time, narrowing 2,315 candidates to 3.19 on average - parity with the expert. Supervised training also moves the correct answer from rank 1,388 under the base model to rank 7.5 out of 12,972 in states that uniquely determine it. The failure was therefore not an inability to reason about constraints.

It was an inability to finish. Given a state with exactly one possible answer, free spelling, and nothing left to decide, the best model names that answer 20% of the time - and 33% even when it had been trained on that exact word. A targeted intervention (Phase 6) that synthesised many endgame paths per word raised unseen-word $k=1$ accuracy from 15.25% to 25.42% ($p = 0.031$), a real and generalising improvement that moved game outcomes by essentially zero. The reason is visible in the data: a 3.46-guess expert almost never visits the endgame, so distilling it supplies thin and early-skewed coverage.

5. Constrained decoding

If the model ranks the right word highly but does not emit it, the remedy is to restrict what it may emit. A word is admissible if and only if it would have produced exactly the feedback already observed. Four decoders were compared on identical games with the same adapter.

Decoder	Mean	Fail	Solved	Invalid	Repeat	HM viol.
unconstrained	5.9634	66.3%	83	21.2%	43.1%	25.3%
legal words only	5.6951	55.7%	109	0%	30.9%	29.4%
legal + no repeats	5.4837	44.3%	137	0%	0%	31.1%
consistent	3.9472	2.0%	241	0%	0%	0%
adaptive	3.7846	2.9%	239	0%	0%	6.8%
classical random	4.0203	0.8%	244	-	-	-
classical frequency	3.7927	1.6%	242	-	-	-
classical entropy	3.4431	0%	246	-	-	-

Table 2. Decoders on the 246 held-out answers, same model throughout. 'HM viol.' is the hard-mode violation rate: guesses inconsistent with feedback already received.

The consistency filter is worth 1.54 guesses on its own - more than everything learned in the three preceding training phases combined. It is the single largest effect in this project.

5.1 Why the filter should not always be on

The expert's own training target is feedback-*inconsistent* most of the time early in the game: at turn 2 only 40.6% of its guesses are consistent, rising to 91.8% at turn 3 and 100% at turn 4. Early on it probes with a word that cannot win. An always-on filter forbids that, which is the known reason hard mode scores worse than free mode. The *adaptive* decoder therefore probes freely while uncertainty is high and becomes consistent once the admissible set falls below a threshold. Sweeping that threshold gives a plateau rather than a peak.

Threshold	Mean	Solved	Fail	HM viol.	Model gain
0 (never filter)	5.6951	109	55.7%	29.4%	1.30
2	4.1707	228	7.3%	16.1%	1.79
5	3.8618	240	2.4%	12.1%	1.70
10	3.7764	243	1.2%	10.2%	1.58
20	3.7642	242	1.6%	9.1%	1.41
50	3.7886	239	2.9%	6.8%	1.21
100	3.8171	239	2.9%	4.5%	1.07
1000 / 1e9 (always)	3.9472	241	2.0%	0.0%	0.57

Table 3. Adaptive-threshold sweep. 'Model gain' is the difference between a no-model control that picks uniformly at random among admissible words and the model's own mean at that threshold. Thresholds 10 to 100 all fall within one standard error of each other; what is outside noise is that both extremes are worse.

5.2 How much is the model and how much is the filter

A decoder this strong invites the objection that the model is a passenger. The control is to play the same games picking uniformly at random among admissible words, with no model at all. That scores 4.5203 under the always-on filter and 5.0027 under the adaptive one, against 3.9472 and 3.7846 with the model - contributions of 0.573 and 1.218 respectively. The model contributes more under the adaptive decoder, because there its learned probing policy replaces a random probe.

An independent check falls out of the design: under the always-on filter, banning repeated guesses is provably a no-op, since a previously played word is inconsistent with its own non-winning feedback. The two configurations returned identical results down to the last game, confirming the filter is applied correctly.

6. Locating the remaining gap

At 3.7642 against the expert's 3.4431, a gap of 0.3211 guesses remains. Two independent measurements localise it, and they answer complementary questions.

6.1 Counterfactual headroom

The first hands one decision regime at a time to the classical expert and replays. This is a hybrid-policy evaluation rather than a replay: substituting an action changes the feedback and every later state, so the model must remain in the loop.

Expert acts in	Mean	Solved	Recovered	% of gap
baseline (model everywhere)	3.7642	242	-	-
2-10 admissible	3.5244	246	0.2398	74.7%
11-100 admissible	3.6707	244	0.0935	29.1%
100+ admissible	4.1626	231	-0.3984	-124.1%
combined (expert everywhere)	3.4431	246	0.3211	100.0%

Table 4. Counterfactual headroom by admissible-set size. The combined row returning exactly the classical entropy mean is a definitional check that the harness is wired correctly. The buckets sum to -0.065 against a combined +0.321, i.e. they are strongly non-additive, because the policies interact at every handoff.

The negative row is instructive rather than paradoxical. An admissible set above 100 means turn 1 or 2, so that condition has the expert play the opening and the model play from turn 3 on - dropping the model into states its training never covered, since it was distilled from a policy with a different opener. It is a policy-mismatch cost, not evidence that the model out-plays the expert.

6.2 The decision budget

The second measurement runs the other way round and is additive. It takes the model's own 922 decisions across the 246 games and prices each against the best action available *at that exact state*, with no substitution and no replay. Where the decoder restricts the action set, the value is computed exactly: the admissible set is small and monotone under filtering, so the remaining decision process is a finite tree solvable to full depth in integer arithmetic.

Admissible-set size	Decisions	Share	Regime
0-1 (forced or solved)	105	11.4%	no choice
2-10	234	25.4%	decoder restricts
11-20	49	5.3%	decoder restricts
21-100	101	11.0%	unrestricted
100+	433	47.0%	unrestricted

Table 5. Where the model's decisions actually occur. 58% are unrestricted, and nearly half are turn-2 probes.

Quantity	Value	Share
Restricted decisions with a real choice	153	100%
Model already optimal	131	85.6%
... where the optimum was a tie	127	83.0%
Genuine mistakes (strictly worse)	22	14.4%
Expected guesses lost to them, total	17.16	-
... per game	0.0698	22% of gap

Table 6. The ceiling on any intervention that only improves action selection. Perfect play across the entire restricted regime - zero mistakes, zero drift - would move 3.7642 to 3.6944.

This single number governs the two experiments that follow. It also rules out on-policy preference mining independently: at 0.089 mistakes per game, all 2,069 training answers would yield roughly 185 usable pairs, and 83% of the decisions the model gets right are ties it could not have got wrong.

7. Preference optimisation

7.1 The run

Direct preference optimisation was run on 14,923 preference pairs aimed at the 2-10 regime, with beta 0.1, learning rate $5e-6$, one epoch, 466 steps. The reference policy was the supervised model itself. Training succeeded on its own terms: loss fell 0.6922 to 0.3988, the implicit reward margin rose from +0.018 to +11.128, and pair accuracy rose from 0.604 to 0.836.

Gameplay regressed. The mean rose from 3.7642 to 3.9350 - paired $t = -3.21$, with 108 games changed, 38 better and 70 worse, and the most common movements being 3 to 4 guesses (28 games) and 4 to 5 (22) against 4 to 3 (18). The model's measured contribution over the no-model control fell from 1.412 to 1.241, confirming the model itself got worse rather than the decoder changing.

7.2 Audit of the preference data

A full audit was run before attempting a second preference experiment. It cleared two hypotheses and found several genuine defects. The preference *direction* was not wrong: re-scored against an exact value function, the labels in the high-value regime were correct on 99.6% of rows and backwards on none. Prompt hygiene was clean.

Defect	Scale
Pairs labelled 'competitive' whose true value gap is large	median 1.00 guesses; only 4.1% within 0.25
'Competitive' pairs separated only by an arbitrary constant	4,072 of 5,367 (75.9%)
Pairs winnable by a feedback-consistency check alone	+44.5 pt asymmetry over 8,479 rows
Budget aimed away from the measured headroom	33.5% of rows in the bucket holding 74.7% of the gap
State duplication	one state contributed 177 rows; 1,867 duplicate ids
Validation split	none existed; the reported accuracy was training accuracy
Held-out answers as the preferred action	694 rows

Table 7. Defects found in the preference dataset. The decisive one is the first: the dataset's own notion of pair difficulty did not track value, so the obvious remedy - keep only the competitive pairs - would have retained 3,499 rows of which 95.9% are not close decisions.

A corrected dataset was built and passed every check: 6,000 training and 800 validation pairs, exact values from an adaptive-decoder tree, one pair per state, no pair between two tied-optimal actions, and no training row naming a held-out answer. It was **never trained on**. The decision budget of Section 6.2 was computed first and showed that the entire available upside, 0.0698 guesses, was smaller than the 0.1708 this method had already been measured to lose. Running it would have been a test of hyperparameters, not of the idea.

7.3 A structural obstacle

Auditing the data surfaced a fact about the task rather than the dataset. Over 1,200 distinct reachable states with 2-10 admissible words, scored exactly: the median state offers six actions of which two are exactly optimal; **95.3% of states have more than one optimal action**, and in 11.9% every action is optimal. The median gap to the best strictly-worse action is half a guess, and only 19.2% of states contain a near-miss within a quarter guess.

A pairwise preference cannot represent 'these two are equally good'. Any generator that simply takes the top two actions by score therefore spends most of its rows asserting an ordering that does not exist. This is the single strongest argument for preferring a group-relative objective here, and it motivates Section 9.

8. Prompt format and lock-in

8.1 The apparent effect

Twelve prompt variants were evaluated with the decoder held fixed. Removing the solver-derived constraint block - the lines listing confirmed letters, present letters, absent letters and ruled-out positions - costs 0.5163 guesses (paired $t = 7.07$). A probe with the decoder switched off is more striking still: without the block the model emits *more* legal words (95.9% against 84.5%) and almost no admissible ones (1.4% against 22.3%). It keeps the rules of Wordle and loses the deduction.

The natural reading is that the harness performs deduction the write-up credits to the model. That reading is not entailed by the data. The adapter had only ever seen one format, so format lock-in predicts the same table. An untrained control cannot break the tie: stock Qwen scores 6.71-6.88 with 91-97% failure and a spread of 0.17 across all twelve variants. A model that cannot play under any prompt cannot rank prompts.

8.2 The crossover

Breaking the tie requires a second model with real skill under a second format. A new adapter was trained on the *same* 19,212 rows with only the prompt text re-rendered, every hyperparameter held identical. The re-render was proved lossless first: each row's parsed history reproduces its stored prompt byte for byte, checked on all 19,212 rows plus the 853 validation rows with zero failures. Training landed on 1,202 optimiser steps, exactly matching the reference run.

Trained on \ evaluated on	baseline	raw_history
baseline	3.7642 242/246	4.2805 232/246
raw_history	4.1098 227/246	3.8089 240/246

Table 8. The format crossover, 246 held-out answers, decoder fixed, all cells opening with the same word. The control cell reproduced 3.7642 exactly.

Comparison	Difference	Paired t	Verdict
Diagonal: each adapter on its own format	+0.0447	+1.08	not significant
baseline adapter moved off its format	+0.5163	+7.07	significant
raw_history adapter moved off its format	+0.3008	+4.92	significant

Table 9. Paired tests on the crossover.

Each adapter is best on its own format and the diagonal is a statistical tie. The prompt penalty was therefore **mostly lock-in**, and the hypothesis that the constraint block encodes deduction a 0.5B model cannot learn at this scale is refuted: a model trained without it recovers to parity.

8.3 The probe disagrees with the score, and both are right

Adapter	Evaluated on	Parse	Legal	Admissible
baseline	baseline	99.3%	84.5%	22.3%
baseline	raw_history	100.0%	95.9%	1.4%
raw_history	baseline	99.3%	93.2%	6.1%
raw_history	raw_history	100.0%	99.3%	0.0%

Table 10. Decoder-off probe over 148 stratified states. The admissible rate is the fraction of raw generations consistent with the feedback already on the board.

The raw_history adapter, evaluated on the format it was trained on, emits **zero admissible words unaided** - and still plays 3.8089. It reaches baseline-equivalent gameplay having learned essentially no feedback consistency, leaning entirely on the decoder. On matched prompts it manages 6.1% against the baseline adapter's 22.3%.

Two true statements point in opposite directions. On final score the constraint block is replaceable: train without it and the decoder covers the difference. On what the model knows, the block is what does the teaching: removing it costs nearly all of the model's unaided deduction. The concern that the harness performs the model's work is not dissolved by this experiment - for the second adapter it is sharpened.

9. Reinforcement learning

9.1 Design

Group-relative policy optimisation was chosen not because it is more powerful than preference optimisation but because it is the only remaining method that reaches the 58% of decisions where the decoder does not restrict the model, and because group-relative advantage represents tied actions natively - the obstacle of Section 7.3.

Rewards were precomputed offline and shipped as data, so the training loop never scores a word; putting a scorer inside the loop would fork the measurement path, the error that voided an earlier result. For states the decoder restricts, values come from the exact adaptive-decoder tree solved to full remaining depth in integer arithmetic. For unrestricted states no exact value is affordable - one lookahead valuation of a single turn-2 state costs about 13.5 seconds at 83 candidates and grows sharply - so those use an exhaustive one-ply expected-remaining value and are labelled as a proxy in the data.

Two departures from the textbook formulation are worth recording. First, because the action set at each state is enumerable (median seven actions) and every reward is precomputed, the exact expectation over the policy is available at the same cost as sampling a group; we use it, and verified numerically that the sampled estimator converges to it as the group grows (maximum gradient-weight error 0.21 at group size 8, 0.003 at 512). Second, advantages are not

divided by the group standard deviation. That normalisation is known to introduce a difficulty bias, and here it is worse than that: with tied optima in 89.5% of the constructed tasks, a group can easily contain only tied actions, yielding a zero-over-zero advantage.

9.2 Result

Training ran 393 optimiser steps over 3,150 tasks. The proxy improved: the optimal-action rate on 380 held-out states rose from 67.6% to 68.9%, and from 77.0% to 78.2% on the exactly-scored subset. No logged step had a degenerate advantage. The run was executed twice, once interactively and once headless on a different machine, producing **byte-identical** adapter weights.

Arm	Mean	Solved	Fail	vs control	Paired t	Games changed
control (supervised)	3.7642	242	1.63%	-	-	-
GRPO, step 150	3.7602	242	1.63%	-0.0041	-0.38	7
GRPO, step 300	3.7520	241	2.03%	-0.0122	-0.90	11
GRPO, final (393)	3.7602	241	2.03%	-0.0041	-0.28	13

Table 11. Gameplay after reinforcement learning, 246 held-out answers, paired. No checkpoint differs significantly from the control. Step 300 has the lowest mean, but at $t = -0.90$ across three comparisons that is selection on noise and is reported rather than adopted.

233 of 246 games were identical. The observed change is 6% of the 0.0698 ceiling and well inside noise. The one real pattern is the last column: games changed rises from 7 to 11 to 13 as training proceeds while the mean stays flat. The policy moves and buys nothing - a smaller, harmless version of the drift that cost the preference run 0.1708. The pre-registered stopping rule, which was to select a checkpoint on the paired rollout rather than on loss, returns the same answer at every step.

10. Discussion

10.1 Two methods, opposite failure modes, one conclusion

Method	What happened	Effect on mean
Direct preference optimisation	drifted away from the reference policy	-0.1708
Group-relative policy optimisation	held its ground, changed almost nothing	+0.0041 (n.s.)

Table 12. The two post-training attempts. Both outcomes were predicted in advance by the decision budget.

Preference optimisation had less upside than its own drift. Reinforcement learning had upside it could not find because there was almost none left. Neither was a botched run. In both cases the ceiling was measured before the method was chosen, and both results landed where the ceiling said they would.

10.2 What the residual gap actually is

The gap is not preference, not ranking, and not prompt format. What remains is the capability limit identified early and never moved: given a state with exactly one possible answer, free spelling, and nothing left to decide, the best model names it 20% of the time. Better action selection cannot fix an inability to produce the right word.

The cause is structural rather than a matter of training budget. A 3.46-guess expert almost never reaches a fully-determined state, so distilling it supplies thin and early-skewed endgame coverage. The intervention that directly targeted this (Section 4.3) improved the measured retrieval ability and still did not move games, which suggests the next lever is a larger model or a fundamentally different source of endgame supervision, not a different objective over the same data.

10.3 Methodological observations

Three practices earned their cost. **Pricing the ceiling before choosing a method** cancelled one fully-built training run whose downside exceeded its entire available upside, and correctly predicted the outcome of the one that did run. **Pre-registering the reading** of an experiment before executing it - done for Sections 8.2 and 9 - made it possible to

report that the crossover matched its predicted numbers and that the reinforcement-learning outcome was the one branch the design had labelled 'flat'. **Keeping a single measurement path** with a control cell that must reproduce a known number caught an adapter-loading error that would otherwise have reported a catastrophic regression: the reinforcement-learning adapter was trained on top of a merged supervised model, and loading it onto the stock base would have silently discarded every supervised weight.

11. Limitations and threats to validity

The unrestricted regime was trained against a proxy. Values there are one-ply expected-remaining over a truncated action menu, not the full vocabulary. This run could teach the model to rank good turn-2 probes but not to avoid bad ones, so 'reinforcement learning cannot help at turn 2' is not established - only that this run did not.

Held-out answers can appear in candidate sets. Training states are generated from training answers, but a large candidate set will contain held-out answers. Training rows never name one as a target, and the rate is reported rather than hidden, but the constraint cannot be enforced at large candidate counts.

246 games is a small evaluation set. The paired protocol is what makes the comparisons resolvable at all; unpaired, the standard error would swamp every effect reported here. Absolute means should be read with that in mind.

The classical reference is not optimal. Greedy entropy sits about 0.03 guesses above depth-6 lookahead, so the 0.3211 gap is measured against a strong heuristic rather than the true floor.

Single model, single task. Every result is for one 0.5B model on one word list. The decision-budget method generalises in principle to any task with a computable value function over an enumerable action set, but that claim is untested here.

12. Conclusion

Distilling a classical Wordle solver into a 0.5B language model produces a system that plays at 3.7642 guesses with 1.63% failure, beating two classical baselines and trailing the expert by 0.3211. Most of that performance is attributable to constrained decoding rather than to supervised learning, though the model contributes a measurable 1.2 guesses over a no-model control under the deployed decoder.

The residual gap resisted four attempts to close it, and the reason is measurable rather than mysterious. Perfect action selection across the regime those methods could reach is worth 0.0698 guesses, and the model is already optimal in 85.6% of the relevant decisions with most of the remainder being exact ties. The gap is a word-retrieval limit inherited from an expert that rarely visits the endgame. We report this as a negative result with a quantitative explanation, which we take to be more useful than a marginal improvement without one.

References

- Anonymous. *Qwen2.5 Technical Report*. Base model: Qwen2.5-0.5B-Instruct.
- Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- Rafailov, R. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023.
- Shao, Z. et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. (Group-relative policy optimisation.)
- Liu, Z. et al. *Understanding R1-Zero-Like Training: A Critical Perspective*. 2025. (Bias introduced by advantage standard-deviation normalisation.)
- Schulman, J. *Approximating KL Divergence*. Blog post, 2020. (The low-variance k3 estimator.)
- Project source, data-generation scripts, audit tools and all result files: github.com/Arnavvs/Qwen_wordle_sft

Appendix A. Reproducibility

All figures in this report come from scripts and notebooks in the project repository with fixed seeds. Two independent checks on determinism were performed. The reinforcement-learning training run was executed twice on different machines, once interactively and once headless, and produced byte-identical adapter weights (SHA-256 prefix 6d4c469a1c1b3790725c). The preference-data generator produces byte-identical output files across runs at the same seed.

The feedback function was validated three ways: against published solver figures for this vocabulary; by rebuilding the full 12,972 x 12,972 feedback table independently and confirming its answer columns are byte-identical to the shipped 12,972 x 2,315 table; and, for the browser demonstration accompanying this report, by checking a JavaScript reimplementation against the Python engine on 408 vectors including adversarial duplicate-letter cases, with 408 of 408 exact.

Appendix B. Principal configurations

Setting	Value
Base model	Qwen2.5-0.5B-Instruct
Adaptation	LoRA rank 16, alpha 32, dropout 0.05, 7 projection modules
Supervised optimisation	lr 2e-4, cosine, 2 epochs, effective batch 16, fp16, seq 640
Supervised data	19,212 rows (7,067 natural, 12,145 synthetic endgame)
Answer split	2,069 train / 246 held out, salted-hash keyed
Deployed decoder	adaptive, filter active when admissible set is 20 or fewer
Preference run	14,923 pairs, beta 0.1, lr 5e-6, 1 epoch, 466 steps
Reinforcement run	3,150 tasks, 393 steps, lr 1e-6, KL coefficient 0.04
Evaluation	246 held-out answers, greedy, paired, six-guess limit
Hardware	single NVIDIA T4

Table 13. Principal experimental configuration.

Appendix C. Summary of all phases

#	Question	Outcome
1	How far does classical solving get?	entropy 3.4644, 0% failure
2	Can the expert produce clean demonstrations?	3 policies x 2,315 games
3	Can they become leak-free supervised data?	7,067-7,173 rows per policy
4	Does supervised fine-tuning work?	No - 76-79% failure
5	Is the failure vocabulary or reasoning?	Neither, mostly
6	Does endgame-heavy data fix it?	Generalises (p=0.031), games unchanged
7	Does a feedback-consistent decoder fix it?	Yes - 3.78
7b	What is the right filter threshold?	Plateau at 10-50; 3.7642
8	Where is the gap; does DPO close it?	74.7% in 2-10 words; DPO regressed
9	Is the harness doing the model's work?	Spread 1.24 guesses, all downside
10	Is that lock-in or the format?	Lock-in ; own-format parity
11	Is the rest of the gap action selection?	No . GRPO +0.004, n.s.

Table 14. The eleven experiments in order.